
POO C#

1. Conceitos Iniciais

1. Classe: É o protótipo utilizado para gerar/instanciar objetos. A classe define atributos e métodos comuns que serão utilizados pelos objetos.

2. Objeto: Objeto é a criação feita a partir da classe.

3. Propriedades: São as variáveis que armazenam os valores de um objeto.

-prop: cria uma propriedade com get e set simples

```
0 references  
public string Nome { get; set; }
```

-propg: cria uma propriedade com get público e set privado

```
0 references  
public int MyProperty { get; private set; }
```

-propfull: cria uma propriedade onde você pode modificar o getter e setter

```
2 references  
private string _nome;  
0 references  
public string Nome  
{  
    get { return _nome; }  
    set  
    {  
        if(value == "") {throw new ArgumentException("Nome vazio");}  
        _nome = value;  
    }  
}
```

4. Método: São as funções da classe

```
0 references  
public int QuantidadeAlunosMatriculados()  
{  
    return Alunos.Count;  
}
```

5. Construtor: Métodos para construção de instâncias

```
1 reference
public Pessoa(String Nome, String Sobrenome)
{
    this.Nome = Nome;
    this.Sobrenome = Sobrenome;
}
```

```
Pessoa p1 = new Pessoa(Nome: "Wesley", Sobrenome: "Freitas");
Pessoa p2 = new Pessoa("Julia", "Fernandes");
```

6. Deconstruct: é um método especial que permite extrair os valores de uma instância e atribuí-los a variáveis individuais. Ele é frequentemente usado em conjunto com tuplas.

- Método Deconstruct criado na classe Pessoa.

```
1 reference
public void Deconstruct(out string nome, out string sobrenome)
{
    nome = Nome;
    sobrenome = Sobrenome;
}
```

- Utilizando o Deconstruct

```
Pessoa p1 = new Pessoa("Wesley", "Freitas");
(string nome, string sobrenome) = p1;
Console.WriteLine($"Nome: {nome}\nSobrenome: {sobrenome}");
```

2. Pilares POO

1. Abstração: Os objetos do mundo real são complexos, por isso é necessário abstrair os atributos essenciais de um objeto, ignorando detalhes irrelevantes.

2. Encapsulamento: Envolve o agrupamento de atributos e os métodos em uma única unidade chamada de classe. O principal objetivo é ocultar a implementação interna de um objeto e fornecer uma interface pública consistente.

- **Definição encapsular:** Ocultar partes da implementação, permitindo construir partes invisíveis ao mundo exterior.

- **Definição interface:** Lista de serviços fornecidos por um componente. É o contato com o mundo exterior.

3. Herança

Herança permite que classes compartilhem atributos e métodos. Ela é usada na intenção de reaproveitar código ou especializar operações.

Obs: Em c# não existe herança múltipla

4. Polimorfismo: Polimorfismo é a habilidade de objetos de classes diferentes responderem a mesma mensagem de diferentes maneiras.

- **Polimorfismo de sobreposição:** Acontece quando substituímos um método de uma superclasse na sua subclasse, usando a mesma assinatura.

- **Polimorfismo de sobrecarga:** Acontece quando temos um mesmo método com assinaturas diferentes na mesma classe.

Construtor de subclasses

Para criar um construtor de uma subclasse, é necessário usar a palavra-chave "base" para chamar o construtor da superclasse.

```
1 reference
public class Aluno : Pessoa
{
    0 references
    public Aluno(string Nome, String Sobrenome, double Nota) : base(Nome, Sobrenome)
    {
        this.Nota = Nota;
    }
    1 reference
    public double Nota { get; set; }
}
```

Polimorfismo na prática

1. Polimorfismo de sobreposição

- É necessário marcar o método da classe mãe como "virtual"

```
1 reference
public virtual void Apresentar()
{
    Console.WriteLine($"Olá, meu nome é {Nome} {Sobrenome}");
}
```

- É preciso marcar o método substituído como "override"

```
1 reference
public override void Apresentar()
{
    Console.WriteLine($"Olá meu nome é {Nome} {Sobrenome} e minha nota é {Nota}");
}
```

2. Polimorfismo de sobrecarga

```
0 references
public Pessoa()
{
}

0 references
public Pessoa(String Nome)
{
    this.Nome = Nome;
}

2 references
public Pessoa(string Nome, string Sobrenome)
{
    this.Nome = Nome;
    this.Sobrenome = Sobrenome;
}
```

Modificadores de acesso

Indicam o nível de acesso aos componentes de um classe

1. Public

A classe atual e todas as outras classes podem acessar.

2. Private

Somente a classe atual pode acessar. Forma de controlar as entradas com set.

3. Protected

A classe atual e todas as suas subclasses podem acessar.

4. Internal

Todas as classes de dentro do mesmo projeto podem acessar.

Classe com listas

Quando uma classe possuir uma lista, é necessário que no construtor a lista seja instanciada, do contrário, quando você for acessar a mesma ocorrerá uma exceção

```
public class Curso
{
    1 reference
    public Curso(){
        Alunos = new List<Pessoa>();
    }
    6 references
    private List<Pessoa> Alunos { get; set; }
```

3. Utilizando o Static

Classe estática

É uma classe que não pode ser instanciada e que contém apenas membros estáticos. Os membros estáticos de uma classe estática podem ser acessados diretamente na classe, sem a necessidade de criar uma instância.

Obs: Classes estáticas são utilizadas para fornecer funcionalidades auxiliares, como bibliotecas de funções, constantes globais entre outras.

Características:

- Todas as propriedades e métodos em uma classe estática devem ser estáticos.
- Membros estáticos são associados à classe, não a instâncias.
- Uma classe estática não pode ter um construtor.
- Não é possível derivar de uma classe estática.

```
public static class Calc
{
    1 reference
    public static void Sum(int a, int b)
    {
        Console.WriteLine(a+b);
        OperacoesRealizadas++;
    }

    2 references
    public static int OperacoesRealizadas { get; set; } = 0;
}
```

<Método estático

É um método que pertence à classe em vez de pertencer a uma instância específica da classe. Isso significa que você pode chamar um método estático diretamente na classe, sem precisar criar uma instância da classe.

```
0 references
public static void Comprimirar ()
{
    Console.WriteLine("Ola");
}
```

```
0 references
public static void Main(string[] args)
{
    Pessoa.Comprimirar();
}
```

4. Utilizando o abstract

<Classe abstrata

Uma classe abstrata tem como objetivo servir apenas como modelo para ser herdado, portanto, não pode ser instanciado.

- Classe abstrata conta

```
1 reference
public abstract class Conta
{
    2 references
    protected decimal Saldo;

    1 reference
    public abstract void Creditar(decimal value);
    0 references
    public void ExibirSaldo()
    {
        Console.WriteLine($"Seu saldo é: {Saldo}");
    }
}
```

- Classe conta corrente que herda de conta

```
0 references
public class ContaCorrente : Conta
{
    1 reference
    public override void Creditar(decimal value)
    {
        Saldo += value;
    }
}
```

<Método abstrato

É um método declarado em uma classe abstrata ou em uma interface, que não tem uma implementação definida. Em vez disso, a implementação do método deve ser fornecida pelas classes derivadas por meio do polimorfismo de sobreposição.

5. Interface

Interface é um recurso muito utilizado para “obrigar” a um determinado grupo de classes a ter métodos ou propriedades em comum, contudo os métodos podem ser implementados em cada classe de uma maneira diferente. Pode-se dizer, que uma interface é um contrato que quando assumido por uma classe deve ser implementado.

Características

- No geral, dentro das interfaces existem assinaturas de métodos e propriedades, cabendo à classe que a utilizará realizar a implementação das assinaturas.
- Uma interface pode ter o corpo de uma função, ficando a critério das classes que a implementam alterar.
- Em c# não existe herança múltipla, então a interface passa a ser uma alternativa.

```
public interface ICalculadora
{
    1 reference
    public int Sum(int a, int b)
    {
        return a+b;
    }
    1 reference
    public int Sub(int a, int b);
    1 reference
    public int Mul(int a, int b);
    1 reference
    public int Div(int a, int b);
}
```

```
1 reference
public class Calculadora : ICalculadora
{
    1 reference
    public int Div(int a, int b)
    {return a/b;}
    1 reference
    public int Mul(int a, int b)
    {return a*b;}
    1 reference
    public int Sub(int a, int b)
    {return a-b;}
    1 reference
    public int Sum(int a, int b)
    {return a+b;}
}
```

6. Comparação entre objetos

1. Comparação Customizada: É necessário implementar a interface `IEquatable<T>` na classe e implementar o método `Equals`.

```
9 references
public class Pessoa:IEquatable<Pessoa>
{
    3 references
    public Pessoa(string Nome)
    {
        this.Nome = Nome;
    }
    4 references
    public string Nome { get; set; }

    0 references
    public bool Equals(Pessoa other)
    {
        if(other == null){return false;}
        return this.Nome == other.Nome;
    }
}
```